Project Ideas

1. Smart Text Pattern Search System

Description:
This project is a text-search engine that efficiently finds a given keyword or pattern in large documents. It applies multiple string matching algorithms and compares their performance.

Key Features:

1. Accept large text input (e.g., text file).
2. Search for single and multiple patterns.
3. Implement **Naïve Search**, **Rabin-Karp**, and **KMP Algorithm**.
4. Display match positions and execution time.
5. Compare efficiency for different input sizes.

Modules / Algorithms Used:

- Input/Preprocessing Module
- Naïve String Matching
- Rabin-Karp Algorithm
- Knuth-Morris-Pratt (KMP) Algorithm

Expected Outcome:
A system that highlights the difference in efficiency between brute force and optimized pattern-matching algorithms for real-world text searching.

2. Data Compression Using Huffman Coding

Description:
This project compresses text files using Huffman coding, reducing file size while maintaining accuracy.

Key Features:

1. Input any text file.
2. Build frequency table of characters.
3. Generate **Huffman Codes** for characters.
4. Compress file and store in binary format.
5. Decompress and reconstruct original file.

Modules / Algorithms Used:

- Frequency Counter Module
- Huffman Tree Construction Module
- Compression Module
- Decompression Module

Expected Outcome:
A working text compressor using greedy algorithm (Huffman coding), with analysis of compression ratio and time complexity.

3. Document Similarity Checker Using LCS

Description:
This project checks similarity between two documents or strings using **Longest Common Subsequence (LCS)** and related dynamic programming approaches.

Key Features:

1. Input two text files.
2. Compute **LCS length** between them.
3. Highlight common subsequences (like plagiarism checker).
4. Display percentage similarity.

Modules / Algorithms Used:

- LCS DP Module

Expected Outcome:
A tool that uses DP to find structural similarity between documents, useful for plagiarism detection or DNA sequence analysis.

---

4. Task Scheduler with Activity Selection

Description:
This project schedules non-overlapping tasks to maximize the number of activities completed, using greedy algorithms.

Key Features:

1. Input tasks with start and end times.
2. Implement **Activity Selection Algorithm (Greedy)**.
3. Compare with a brute-force approach.
4. Show visualization of scheduled tasks.
5. Output maximum number of tasks completed.

Modules / Algorithms Used:

- Greedy Scheduler Module
- Brute Force Scheduler
- Time Complexity Analyzer

Expected Outcome:
An efficient scheduler that highlights how greedy algorithms outperform brute force in interval scheduling.